

Article

Evaluating YOLO Models for Efficient Crack Detection in Concrete Structures Using Transfer Learning

Muhammad Sohaib ¹, Muzamal Arif ² and Jong-Myon Kim ^{3,4,*} 

¹ School of Computer Science and Technology, Zhejiang Normal University, Jinhua 321004, China; sohaibdurrani@zjnu.edu.cn

² Department of Computer Science & Software Engineering, Grand Asian University, Sialkot 51040, Pakistan; muzamil.arif@gaus.edu.pk

³ Department of Electrical, Electronics and Computer Engineering, University of Ulsan, Ulsan 44610, Republic of Korea

⁴ Prognosis and Diagnostics Technologies Co., Ltd., Ulsan 44610, Republic of Korea

* Correspondence: jmkim07@ulsan.ac.kr

Abstract: The You Only Look Once (YOLO) network is considered highly suitable for real-time object detection tasks due to its characteristics, such as high speed, single-shot detection, global context awareness, scalability, and adaptability to real-world conditions. This work introduces a comprehensive analysis of various YOLO models for detecting cracks in concrete structures, aiming to assist in the selection of an optimal model for future detection and segmentation tasks. The YOLO models are initially trained on a dataset containing both images with and without cracks, producing a generalized model capable of extracting abstract features beneficial for crack detection. Subsequently, transfer learning is employed using a dataset that reflects real-world conditions, such as occlusions, varying crack sizes, and rotations, to further refine the model. Crack detection in concrete remains challenging due to the wide variation in crack sizes, aspect ratios, and complex backgrounds. To achieve optimal performance, we test different versions of YOLO, a state-of-the-art single-shot detector, and aim to balance inference speed and mean average precision (mAP). Our results indicate that YOLOv10 demonstrates superior performance, achieving a mean average precision (mAP) of 74.52% with an inference time of 19.5 milliseconds per image, making it the most effective among the models tested.



Citation: Sohaib, M.; Arif, M.; Kim, J.-M. Evaluating YOLO Models for Efficient Crack Detection in Concrete Structures Using Transfer Learning. *Buildings* **2024**, *14*, 3928. <https://doi.org/10.3390/buildings14123928>

Academic Editor: Alberto Taliervo

Received: 5 November 2024

Revised: 25 November 2024

Accepted: 4 December 2024

Published: 9 December 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: crack detection; concrete structures; structural health monitoring; transfer learning YOLO

1. Introduction

Concrete is the virtual workhorse of civil engineering, underpinning bridges, buildings, roads, and pavements [1], yet even this ubiquitous material is not impervious to the havoc of time. Over the passage of time, it slowly deteriorates due to various causes, including environmental effects, aging, fatigue, disproportionate loading conditions, and the gradual degradation of construction elements [2–5]. The detection of cracks is vital as it suggests the early stages of structural degradation, substantiating peril to the endurance and resilience of the structure [6]. Further cracking can be caused by the repeated action of freezing and thawing, which puts additional stress on the already fragile concrete [7]. If left unchecked, these issues can culminate in the spalling, the chipping away of concrete bits from the surface, and the eventual complete crumbling of the material [8]. Cracks allow the flow of water and hazardous substances into a concrete structure, which results in corrosion, spalling, and disintegration of the structure [9]. Such issues threaten public safety and undermine structural integrity. When concrete fails, it can result in injuries or fatalities, as well as the destruction of properties and nearby facilities.

Traditionally, a manual visual procedure is used to inspect concrete structures for the manifestation of cracks [10]. However, it is hectic and time-consuming and requires

significant domain expertise. Additionally, crack detection relies solely and directly on the inspector so the experience level of the inspector determines the level of crack detection accuracy. To tackle this issue, non-invasive techniques are exercised to identify and segment cracks in concrete structures incorporating image processing, machine learning, and deep learning techniques [11–13]. These non-invasive and automated techniques are faster, economical, and secure for human inspection. These non-invasive techniques build up on the pillars of image processing, machine learning, and deep learning.

In addition to these advancements, radar-based crack detection has emerged as a valuable non-invasive technique [14]. Ground-penetrating radar (GPR) can penetrate materials and detect subsurface defects that are otherwise invisible to visual methods, making it a useful complement to traditional inspection [15]. However, radar data often require manual interpretation due to the complexity and variability of the signals, limiting full automation.

The integration of advanced image processing and machine learning techniques holds promise for reducing manual intervention, improving accuracy, and automating the detection process. Such innovations are critical as they combine the benefits of both surface and subsurface crack detection into a comprehensive monitoring solution. The existing image processing algorithms are capable of improving the quality of the visual data and thereby highlighting even fine cracks [16]. Crack data can also be electronically stored as images with labels and a large amount of labeled crack images can be used to train machine learning algorithms to increase recognition and categorization accuracy over time [17]. This is taken a step further in the subfield of machine learning known as deep learning, which employs deep artificial neural networks to deliver even higher levels of crack detection accuracy [18].

Scientists are leveraging the capabilities of deep learning algorithms to find novel and self-sufficient approaches. In this aspect, Bhattacharya et al. introduced a deep learning-based approach for defect identification in concrete structures. The designed network was termed an interleaved deep artifacts-aware attention mechanism. The proposed approach was used to classify images associated with structural defects [17]. Additionally, a feature pyramid network (crack-FPN) was proposed for crack detection and segmentation [19]. The proposed approach consisted of You Only Look Once version 5 (YOLOv5) and the feature pyramid network (FPN). It was able to effectively detect and segment cracks but the inference time was relatively high. Likewise, Zhang et al. [20] proposed an effective crack detection mechanism in concrete structures using a broad learning-based MobileNetv3 model. Moreover, a convolutional neural network (CNN) integrated with XGBoost and random forest (RF) was suggested for concrete crack detection [21]. Although this approach demonstrated high performance on certain data, its efficacy deteriorated on unseen samples.

While deep learning has made significant strides in concrete crack detection, a major hurdle still remains in the detection of concrete cracks, i.e., crack detection in complex background images. In some of these works, optimal performances are demonstrated on image databases that contain simple images with no background distractions [13], but real-life settings are not as accommodating. These crack detection techniques can be hampered by background shadows and abrasions such as debris, among others. Adding to the complexity, some of the models that rely on deep learning are challenged with a new set of unseen data. The model that works well when trained on a certain set of images might fail when tested on the actual image that has some elements in the background that the model has not encountered before [22]. Additionally, the speed of inferring cracks in real time is also a major concern in the field. To perform crack detection and segmentation in real time, researchers have proposed different algorithms [23]. Different variants of YOLO have shown accurate and precise detection and segmentation of cracks even in challenging environments with high inference speed [24]. Based on the characteristics of YOLO, the designed crack detection and segmentation model has a few advantages over the other models. The different YOLO architecture works on the basis of a single-shot detection mechanism. Therefore, it yields outputs significantly faster as compared to other object

detection models. The newer versions of the YOLO network are optimized to provide a trade-off between accuracy and speed. Additionally, the YOLO architectures are aware of the global context of the input image. This property makes small objects in challenging environments easily detectable by these networks. Furthermore, these architectures are easily scalable as per the needs of underlining hardware recourses. It can also handle various object scales and aspect ratios. These characteristics of YOLO-based crack detection and segmentation networks make them suitable for real-time scenarios. Despite significant advancements in crack detection using deep learning, existing models struggle with detecting cracks in complex background environments and fail to generalize well when faced with unseen data. Additionally, there is a lack of comprehensive benchmarking of different YOLO variants specifically for crack detection and segmentation, particularly in real-world settings with complex backgrounds. This work provides an empirical study of the performance of different YOLO models in detecting and segmenting cracks. A two-stage transfer learning mechanism is applied to all the variants of the YOLO architectures considered in this study to enhance the generalization power of the designed models. The aim of the study is to identify the variant of the available standard YOLO architectures that can detect and segment cracks with high speed, accuracy, and precision. This work can provide insight into the selection of a suitable YOLO model for future research work on this topic in real-time scenarios. The contributions of this work are stated as follows:

1. A two-stage transfer learning approach is applied to the different YOLO models to enhance their generalizability;
2. A detailed comparison of different YOLO models is presented on different crack detection and segmentation datasets.

2. Monitoring Structural Giants in Urban Landscapes

This includes overarching bridges, tall buildings, or other large projects that take decades to complete. However, as time passes, these structures start to have what is commonly referred to as ‘cracks’, which may extend and affect the stability of the concrete infrastructure, thus putting the general public at risk. To prevent the failure of concrete and to guarantee public safety, there is a need for a reliable system for the early detection of these cracks. For this purpose, several supervised and unsupervised machine learning and deep learning methods for crack detection and segmentation have been proposed for this purpose, some of which are YOLO-based models. The current YOLO-based crack detection models are ideal for real-time applications.

While there is no detailed analysis of the different configurations of YOLO, this study aims to compare the various versions of the network in their basic setups. The objective is to determine which of the available YOLO architectures is capable of detecting and segmenting the cracks with the fastest speed, highest accuracy, and least measurement error. This research can be beneficial in recommending the most suitable YOLO model for subsequent research with real-time crack detection.

In the comparative assessment of various YOLO models, the transfer learning two-stage network is applied for detecting the cracks on concrete surfaces, ranging from the macro to the micro level. Thus, given the architecture and the operation of various versions of YOLO, the task of creating effective tools for the diagnostics and safety of concrete constructions becomes achievable. It is the specifics of the methodologies employed as a technical premise for this study that are described in the subsequent section.

2.1. Deep Neural Networks and the Challenge of Depth

Crack detection in concrete structures is one of the areas that greatly benefits from the use of deep neural networks (DNNs), especially when detailed information needs to be extracted from images. A key advantage of these networks is their depth—the number of layers in the network—which allows them to capture intricate details. However, deeper networks encounter issues such as the vanishing gradient problem, which slows down the learning process.

Training deep networks requires optimizing weights and biases through backpropagation, where error signals propagate from the output layer back to the input layers to adjust the model's predictions. In very deep architectures, this error signal becomes smaller as it moves through more layers, making learning difficult in the earlier layers. This is known as the vanishing gradient problem, and it significantly hampers a network's ability to learn complex features, reducing its capacity to capture the finer details needed for crack detection.

Several methods have been developed to address this issue. Architectures like Residual Neural Networks (ResNets) use shortcut connections, enabling the training of much deeper models. These deeper models can learn both macro- and micro-level features in images, making them especially useful for detecting small cracks in concrete structures.

While many architectures, including CNNs and transformer-based networks, have been applied to crack detection, specific architectures stand out due to their balance between performance and efficiency. YOLO (You Only Look Once) is one such model that has gained attention for its ability to detect objects in real time with high precision. The newer versions of YOLO are designed to work with deeper networks, effectively addressing issues like the vanishing gradient problem. These models are particularly useful for on-site crack detection, where immediate performance is critical.

In summary, combining highly effective training techniques, such as residual connections, with efficient architectures like YOLO makes these networks ideal for practical structural health monitoring processes.

2.2. Double Transfer Learning: A More Focused Approach

Transfer learning methods have long been popular in deep learning, enabling models to apply knowledge gained from one task to another. However, when it comes to crack detection and segmentation in concrete structures, standard transfer learning techniques often fall short. Generic pre-training tasks, such as image classification on large datasets like ImageNet, fail to equip the model with the specific features necessary for accurately identifying cracks. The result is that the model may struggle to distinguish the subtle visual patterns associated with cracks in concrete surfaces.

This is where Double Transfer Learning (DTL) offers a more focused and effective solution, especially when integrated into architectures like YOLO. DTL introduces a two-step transfer learning protocol, designed to bridge the gap between the "universal" features learned during pre-training and the specialized features needed for crack detection. By incorporating an intermediate task closely aligned with crack detection, DTL ensures that the model is fine-tuned with more relevant information before being applied to the final task.

In our comparative study of different YOLO architectures, we found that DTL significantly improves the performance of these models on crack detection and segmentation datasets. The double transfer approach enhances the models' ability to learn both coarse and fine-grained features, allowing for more accurate and efficient crack detection. The first stage of transfer learning leverages large-scale general-purpose datasets to establish a robust foundation. The second stage then fine-tunes the model on a crack-specific dataset, honing the architecture's ability to detect cracks at both macro and micro levels.

The application of DTL across different YOLO versions demonstrated that this method not only boosts detection accuracy but also improves the models' ability to segment cracks in real time, a critical factor for practical on-site applications. This two-step process ensures that the models are better prepared to handle the nuances of the task, making DTL an ideal approach for enhancing crack detection in concrete structures.

The following sections will explore the technical details of different YOLO architectures and how DTL is implemented in more detail, along with a comparison of performance metrics across different YOLO architectures.

3. Overview of Various YOLO Models

3.1. Overview of YOLOv5

YOLOv5 is an object detection model that is part of the YOLO series, which was developed by Ultralytics [25]. The basic architecture of YOLOv5 is given in Figure 1. It leverages the previous versions known as YOLOv1, YOLOv2, YOLOv3, and YOLOv4 but adds a number of enhancements in many ways starting with efficiency. YOLOv5 is also implemented in the PyTorch $\geq$ 1.7 format, thus allowing for easier extension and modification of the model than the previous versions that were implemented in Darknet.

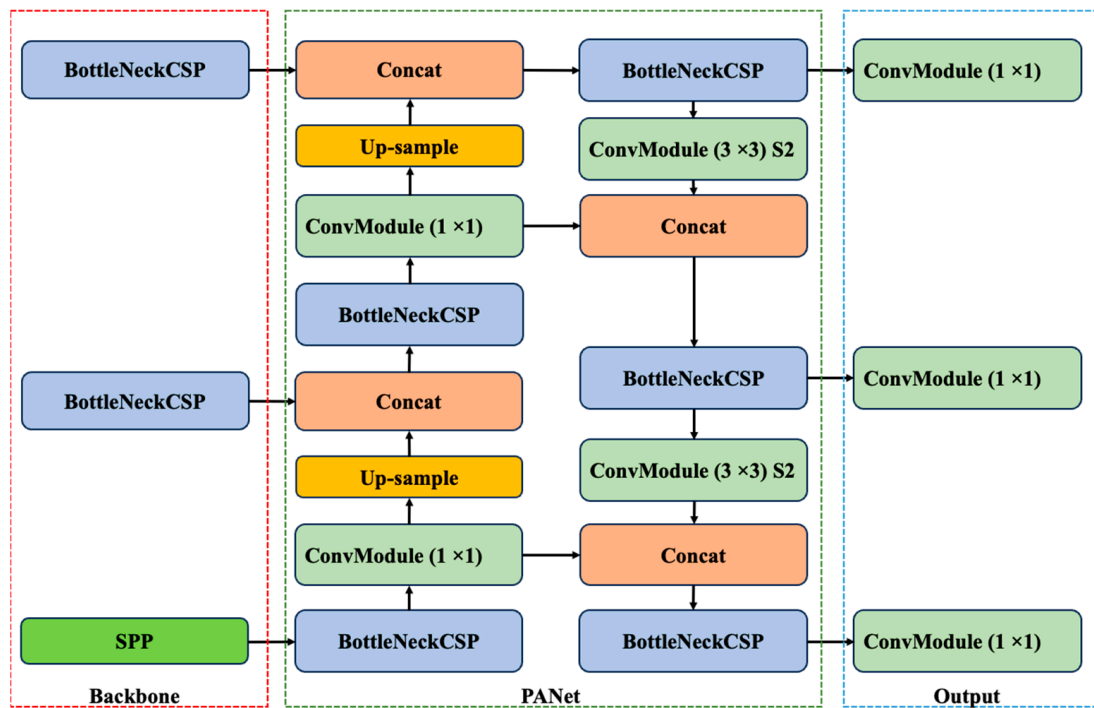


Figure 1. An illustration of a standard YOLOv5 network [26].

Even though the creation of YOLOv5 was not by the original creator Joseph Redmon, it is one of the most popular versions owing to its practical improvements, efficiency, and flexibility in real-world situations.

3.2. Key Modules of YOLOv5

3.2.1. Backbone: CSPDarknet53

The backbone is designed to produce feature maps of inputs or the input images in this case. The backbone of YOLOv5 is CSPDarknet53, also called Cross Stage Partial Darknet53. This backbone is something like Darknet53, which was used in the YOLOv4 model but has better gradient flow, which makes it fast in terms of computation and feature extraction. The CSP (Cross Stage Partial) network divides the feature map into two and combines the map again to make the learning efficient.

3.2.2. Neck: PANet

The neck of YOLOv5 plays an important role in enabling the fusion of feature maps from different levels in the backbone. It employs the Path Aggregation Network (PANet) and FPN (Feature Pyramid Network) in this context. These networks are intended for effective data flow and feature extraction at different resolutions crucial for the detection of objects of different sizes.

FPN (Feature Pyramid Network): Improves the chances of identifying small objects from the feature maps constructed from low-level features of an image.

PANet: Enhances the localization of objects as well as the passing of the low levels of features from the backbone to the head.

3.2.3. Head

Originally, the final object detection task was also included in the head of YOLOv5, which involves bounding box regression and classification. The detection head works with three anchor-based predictions at three scales (small, medium, and large) based on anchors. The anchor box outputs the object class, the four coordinates of the bounding box, and an object confidence score.

3.3. Overview of YOLOv6

The architecture of YOLOv6 given in Figure 2 introduces several novel components and improvements across three main parts, including spines also known as backbones, necks, and heads [27]. In real-time detection tasks, the model should be highly efficient while the speed must be slightly compromised for better detection accuracy.

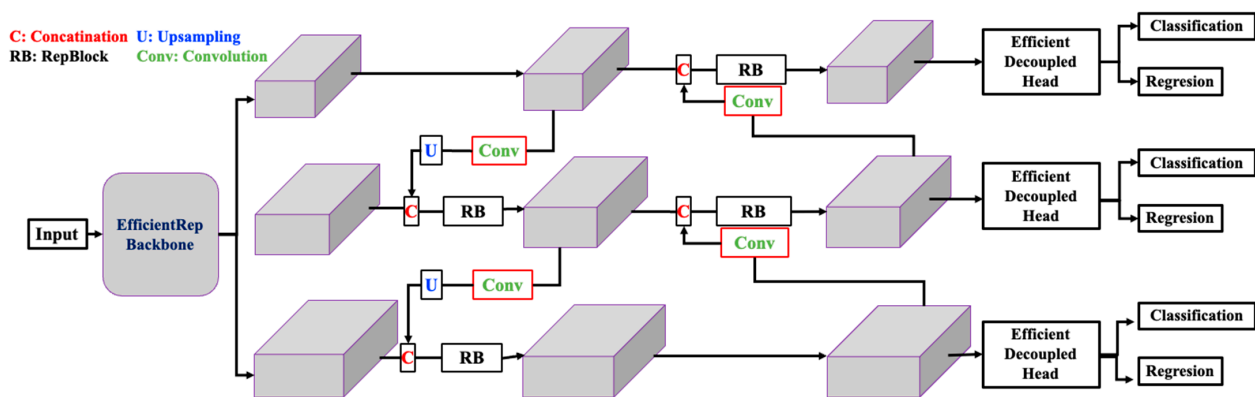


Figure 2. An illustration of a standard YOLOv6 network [27].

3.4. Key Modules of YOLOv6

3.4.1. Backbone: EfficientRep

The core of YOLOv6 is EfficientRep, which is a fast and strong feature extractor based on RepVGG. As can be seen, RepVGG has a basic structure, consisting of a stack of 3×3 convolutions, although it is well-tuned for hardware optimization. EfficientRep improves upon RepVGG in the way that it is better designed for feature extraction in the particular case of object detection. It makes use of RepBlocks, which consist of both training-time as well as inference-time representations in order to minimize computational complexity while maintaining performance.

RepVGG Blocks: The convolution blocks are simple and efficient with reparameterization along with better performance during inference.

Fewer Parameters: A lightweight backbone that results in a considerable reduction in parameters compared to the prior YOLO models while enhancing the speed of YOLOv6 without a dire effect on the accuracy.

3.4.2. Neck: PAN Namely Path Aggregation Network

The neck of YOLOv6 is equipped with an improved Path Aggregation Network (PAN) that makes it possible to integrate features from different scales to help detect objects of different sizes. The PAN neck aggregates feature maps from the backbone, which makes it possible for it to combine low-level and high-level information.

Multi-scale feature fusion: PAN makes sure that features of an object from both coarse and fine resolution are integrated and can detect small, medium, and large objects easily.

Bidirectional Feature Pyramid Network (BiFPN): New to YOLOv6, the network is an efficient BiFPN for improved feature combinations that can help increase the multi-scale detection rate while reducing computational costs.

3.4.3. Head: Decoupled Head

YOLOv6 features a Decoupled Head, which separates classification and regression tasks into two branches. This design improves detection accuracy by focusing separately on bounding box regression and object classification, reducing interference between the two tasks.

Bounding Box Regression Head: Focuses on predicting the exact coordinates and size of the bounding box.

Classification Head: Responsible for determining the object class within each detected bounding box. This decoupling allows for more specialized learning, improving performance for both tasks.

3.5. YOLOv6 Variants

YOLOv6 comes in several variants, optimized for different use cases by balancing between speed and accuracy. These variants follow a naming convention similar to YOLOv5, with increasing size and complexity:

YOLOv6-N: Nano version, optimized for extremely fast inference with minimal computational resources.

YOLOv6-S: Small version, a balance between speed and accuracy, suitable for low-power devices.

YOLOv6-M: Medium version, offering better accuracy with a moderate increase in computational cost.

YOLOv6-L: Large version, designed for higher accuracy at the cost of speed, suitable for higher-end hardware.

YOLOv6-X: Extra-large version, providing the highest accuracy but requiring the most computational resources.

3.6. Overview of YOLOv8

The architecture of YOLOv8 has evolved significantly to support higher accuracy and computational efficiency. As given in Figure 3 the core components include the backbone, neck, and detection head, with improvements in all areas [28].

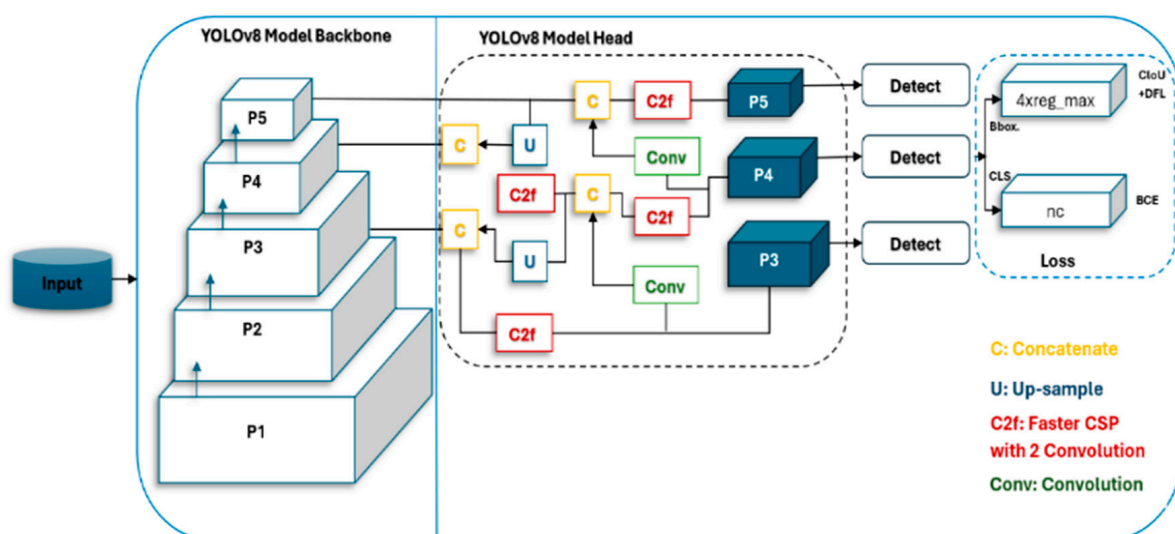


Figure 3. An illustration of a standard YOLOv8 Network [28].

3.7. Key Modules of YOLOv8

3.7.1. Backbone: Variant of CSPDarknet

The backbone in YOLOv8 is responsible for extracting features from the input image. YOLOv8 incorporates a CSPDarknet-based backbone with enhanced residual connections, focusing on reducing computational complexity while improving feature extraction.

CSP (Cross Stage Partial Networks): YOLOv8 employs a CSP structure to improve the gradient flow and reduce the number of parameters. This helps in training deeper networks without the vanishing gradient problem.

Squeeze-and-Excitation (SE) Blocks: SE blocks are added to help the model focus on relevant features by re-weighting channel-wise features, allowing YOLOv8 to learn more discriminative features.

Residual Connections: The backbone includes residual connections (inspired by ResNet), allowing the model to train deeper layers efficiently while maintaining the flow of gradients.

3.7.2. Neck: PANet (Path Aggregation Network)

YOLOv8 uses PANet as the neck, which is responsible for merging multi-scale features from different stages of the backbone. This allows the model to detect objects at different scales (small, medium, large).

FPN (Feature Pyramid Network): A hierarchical structure that enables multi-scale feature extraction, allowing YOLOv8 to perform better at detecting small objects.

PAN (Path Aggregation Network): Enhances feature fusion by combining lower-level features with higher-level features, improving object detection accuracy across different object sizes.

3.7.3. Detection Head: Decoupled Head

The detection head is responsible for generating the final predictions, including object classification and bounding box regression. YOLOv8 introduces a Decoupled Head, separating the tasks of localization (bounding box prediction) and classification.

Decoupled Classification and Regression: The Decoupled Head allows YOLOv8 to handle the regression of bounding boxes and the classification of object categories independently, reducing interference between the two tasks. It uses Complete Intersection over Union (CIoU) and Distributional Focal Loss (DFL) [29,30] to estimate the regression loss of the bounding boxes.

Anchor-Free Design: YOLOv8 introduces an anchor-free mechanism, eliminating the need for predefined anchor boxes. This simplifies the architecture and improves efficiency, especially for edge devices. The model directly predicts the center, width, and height of the object.

3.8. Variants of YOLOv8

Similar to YOLOv5, YOLOv8 offers several variants with increasing complexity to balance between speed and accuracy. These variants allow the model to be deployed across a wide range of hardware and application environments.

YOLOv8n (Nano): The fastest lightweight model with the lowest accuracy but ideal for low-powered devices.

YOLOv8s (Small): A balanced model suitable for real-time applications requiring both speed and moderate accuracy.

YOLOv8m (Medium): Offers better accuracy with a slight increase in computational requirements.

YOLOv8l (Large): Designed for more powerful hardware, providing higher accuracy.

YOLOv8x (Extra-Large): The most accurate but computationally expensive model, used in scenarios where precision is more critical than inference speed.

3.9. Overview of YOLOv9

YOLOv9 is a computer vision architecture designed that includes capabilities for object detection and image segmentation [31]. YOLOv9 goes a notch higher than its predecessors incorporating revolutionary concepts such as PGI and GELAN to enhance the speed and accuracy of object detection tasks. An illustration of the standard YOLOv9 model is shown in Figure 4.

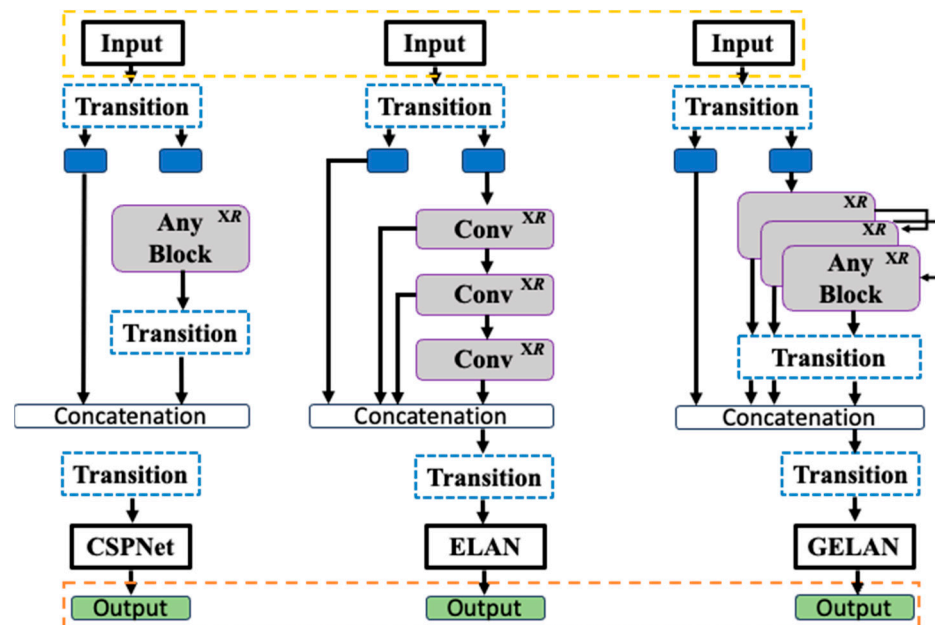


Figure 4. An illustration of a standard YOLOv9 Network [31].

With the integration of PGI, YOLOv9 overcame the problem caused by the deep networks such that there is no data loss and the gradients generated for training are accurate. The application of GELAN results in a network structure that optimizes the parameters together with an efficient computational rate and thus makes the YOLOv9 flexible as well as highly efficient for use in varied applications. In the following section, we will delve into four essential aspects of YOLOv9. We begin with an overview of the Information Bottleneck Principle, which provides the necessary context, followed by the three techniques used to address these bottlenecks: reversible functions, PGI, and GELAN.

3.10. Background: Information Bottleneck Principle

The Information Bottleneck Principle outlines the process of information compression that occurs as data undergo transformations within a neural network. This concept is mathematically represented by the Information Bottleneck equation, which reduces the mutual information between the original input data and its transformed state as it progresses through the various layers of the deep network. The Mathematical Expression describing Information Bottleneck Principle is given below [24]:

$$M(I, I) \geq M(I, f_{\phi}(I)) \geq M(I, g_{\theta}(f_{\phi}(I))) \quad (1)$$

In this equation, M symbolizes mutual information that has nonlinear transformation functions f and g with parameters ϕ and θ , respectively. While passing through the two layers of a deep neural network, f_{ϕ} and g_{θ} , respectively, data I are inevitably stripped of some important information for making accurate predictions. This loss may lead to instabilities in gradients and difficulties in the convergence of the model.

One solution is to make the model larger to increase its throughput capabilities and thus maintain more of the data content. But, this has not solved the issue of treatment of

gradients in very deep networks, which is very irregular. The subsequent section of the analysis looks at how reversible functions provide a more feasible option.

The Information Bottleneck Principle explains how information is compressed as data undergo transformations within a neural network. This principle is captured by the Information Bottleneck equation, which simplifies the mutual information between the original input data and its transformed representations as it moves through the layers of a deep network.

There are several developments with YOLOv9: firstly, reversible functions deal with information bottlenecks; secondly, PGI enhances the accuracy of the model; and thirdly, GELAN derives benefits from the Petabyte-Scale Image dataset.

3.11. Key Modules of YOLOv9

3.11.1. Reversible Functions

The Information Bottleneck is a theoretical disease while the reversible function is the theoretical cure. Enshrined within neural networks, reversible functions ensure zero loss of information degradation during the different operations performed on data. These functions keep all of the transformations in the network in a state, which allows the exact reconstruction of the input data to be accomplished from the output values. The reversible function equation is stated as follows [24].

$$I = v_{\zeta}(r_{\psi}(I)) \quad (2)$$

3.11.2. Programmable Gradient Information (PGI)

The practice of having reversible functions in different neural networks brings a new paradigm in deep neural network training that will not only give dependable gradient for model updates but also incorporate shallow and lightweight neural networks.

Programmable Gradient Information appears as a solution in the form of a main forward-pass branch, a reversible side branch for accurate gradient calculation, and multiple levels of helpful side information to overcome deep supervision issues without incurring extra inference costs.

The information listed below divulges the fact that the YOLOv9 framework is designed with the purpose of enhancing the training of the model and has many layers of Programmable Gradient Information embedded within it. PGI includes another supervision node for auxiliary that is designed to overcome the information bottleneck issue in deep neural networks while keeping an eye on the accurate and efficient computation of gradient backpropagation. PGI changes in the course of its development as it is based on three components, each of which has its own function within the structure of the PGI model but at the same time interacts with the other components.

Main Branch: Made for inference only, the main branch helps keep the model and its operation simple and free from unnecessary operations, especially during important steps of the process. Designed in a way as to remove the need for additional components during inference while maintaining high efficiency, it does not add extra computational loads.

Auxiliary Reversible Branch: Relative to the above aspects, the auxiliary branch ensures that reliable gradients are generated and appropriate modifications to parameters are attained. By using reversible architecture, it reduces the inherent problem of information loss deep in some of the network layers and makes whole information available for learning. Inference speed and essentially both depth and complexity of models cannot infringe this branch's modularity, which allows it to reduce or expand successfully.

Multi-Level Auxiliary Information: This methodology employs particular networks to aggregate gradient information across all the layers of the model. It also caters to the problem of information loss in deep supervision models so that the data are effectively understood by the model. It also increases the model's reliability in predicting the size of objects of different dimensions.

3.11.3. Generalized Efficient Layer Aggregation Network (GELAN)

This further application of the PGI in the YOLOv9 architecture leads to a rather logical request for the formation of an even more delicate design to achieve the maximum accuracy of predictions. This is where the Generalized Efficient Layer Aggregation Network (GELAN) comes in.

GELAN brings in a novel design to fit the PGI framework; this improves the ability of the model to analyze and learn from information. Whereas PGI deals with one of the key issues of maintaining crucial data across deep neural networks, GELAN takes it a step further by providing a modular effective framework that can incorporate various computational components.

In YOLOv9, the GELAN encompasses the best of both CSPNet's gradient path planning and ELAN when it comes to inference speeds. This flexible architecture ensures these characteristics, enhancing the real-time inference that defines the YOLO family. Here, GELAN can be viewed as a lightweight framework that promotes fast inference with accuracy, thus expanding the applicability of computational blocks.

3.12. Overview of YOLOv10

YOLOv10, given in Figure 5, represents an advanced evolution of the original YOLO object detection models, featuring an improved architecture that enhances both accuracy and efficiency [32]. The YOLOv10 design incorporates a lightweight classification head utilizing depthwise separable convolutions, which reduces computational demands without sacrificing performance. Additionally, YOLOv10 decouples spatial downsampling from channel transformation, further enhancing efficiency. A notable innovation in YOLOv10 is its rank-guided block design, which identifies and replaces redundant stages with a compact inverted block structure, optimizing performance while lowering computational costs. The key features of YOLOv10 are as follows:

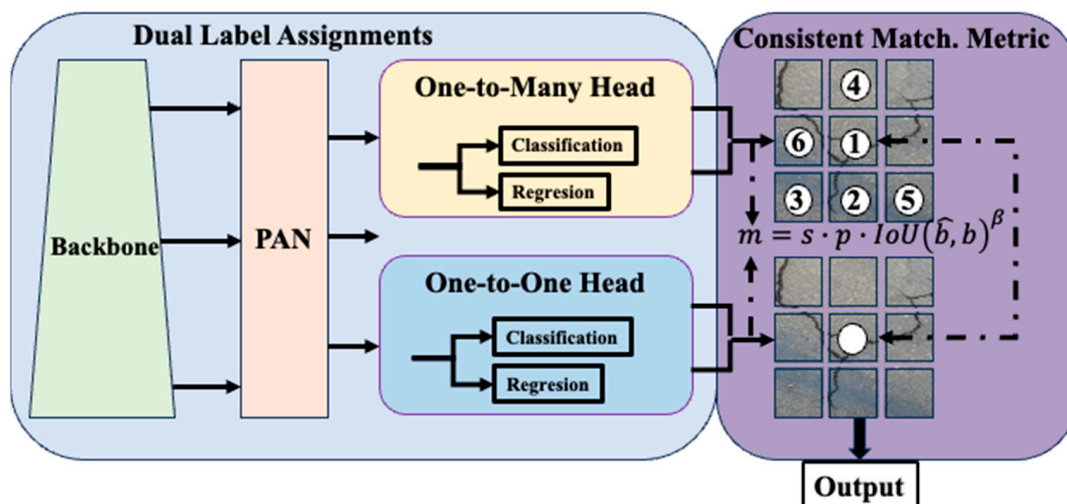


Figure 5. An illustration of a standard YOLOv10 Network [32], where numbers 1–6 represent the multiple predictions per object during training to provide rich supervisory signals.

NMS-Free Training Strategy: YOLOv10 employs a dual assignment strategy that eliminates the need for non-maximum suppression.

Consistent Dual Assignments Approach: This strategy integrates one-to-one and one-to-many matching techniques, enriching the supervisory signals during training and improving overall performance.

One-to-One and One-to-Many Matching: YOLOv10 assigns a single prediction to each ground truth instance, eliminating the necessity for non-maximum suppression during inference. The one-to-many assignment provides additional supervisory information.

3.13. Key Modules of YOLOv10

3.13.1. Lightweight Classification Head

In the YOLOv10 model, the new lightweight classification head design enhances computational performance. This innovation represents a significant step forward in improving model efficiency and addressing key challenges in real-time object detection. Compared to previous YOLO models, the YOLOv10 classification head achieves a considerable reduction in FLOPs (floating-point operations per second) and parameter count, leading to improved efficiency.

For instance, YOLOv10-B has 46% lower latency and 25% fewer parameters than YOLOv9-C, while maintaining comparable accuracy. Additionally, YOLOv10-S demonstrates an impressive speed improvement, running eight times faster than the RT-DETR-R18 model with $8\times$ downsampling, while maintaining a similar mean average precision (mAP) on the COCO dataset. Furthermore, it achieves this using eight times fewer parameters and FLOPs.

Overall, YOLOv10 underscores the importance of creating a lightweight model that successfully balances speed and accuracy, making it ideal for real-time applications.

3.13.2. Spatial-Channel Decoupled Down-Sampling

YOLOv10 introduces a revolutionary down-sampling approach by separating spatial and channel operations. This technique improves feature extraction while reducing computational costs. Unlike previous models that combined spatial and channel transformations through 3×3 convolutions with a stride of 2, YOLOv10 first adjusts the channel dimension using pointwise convolution and then applies depth-wise convolution for spatial down-sampling. This separation reduces the number of parameters and preserves more information, resulting in competitive performance and reduced latency. Key advantages of this method include

1. Enhanced feature extraction optimization;
2. A significant reduction in computational cost;
3. Improved information retention;
4. Faster inference times.

This spatial-channel decoupled down-sampling method marks a significant advancement in object detection model efficiency without compromising accuracy.

3.13.3. Rank-Guided Block Design

YOLOv10 features an innovative rank-guided block design that enhances neural network optimization and model stage refinement. This approach establishes a new standard in object detection.

Intrinsic Rank Analysis: YOLOv10's development involved an intrinsic rank analysis to identify redundancies in the model's stages, revealing that uniform block designs across all stages were not optimal.

Compact Inverted Block Structure: In response, YOLOv10 introduced the Compact Inverted Block (CIB) structure, which integrates depthwise convolutions for spatial mixing and pointwise convolutions for channel mixing. This structure is part of the Efficient Layer Aggregation Network (ELAN), improving overall efficiency.

Optimization of Model Stages: YOLOv10 employs a rank-guided block allocation strategy to replace redundant stages with more efficient designs, ensuring that performance remains high while computational costs are minimized.

This rank-guided block design significantly enhances YOLOv10's computational efficiency and sets a new benchmark in object detection technology.

4. Methodology

This research employs digital images acquired through 2D sensors (cameras) and different versions of deep learning algorithms, namely YOLO to detect and segment cracks

in concrete structures. The aim of this research is to provide insight into the selection of a suitable YOLO model for future research work on this topic in real-time scenarios. The strategy to develop and deploy in the experiment consists of the following phases as shown in Figure 6.

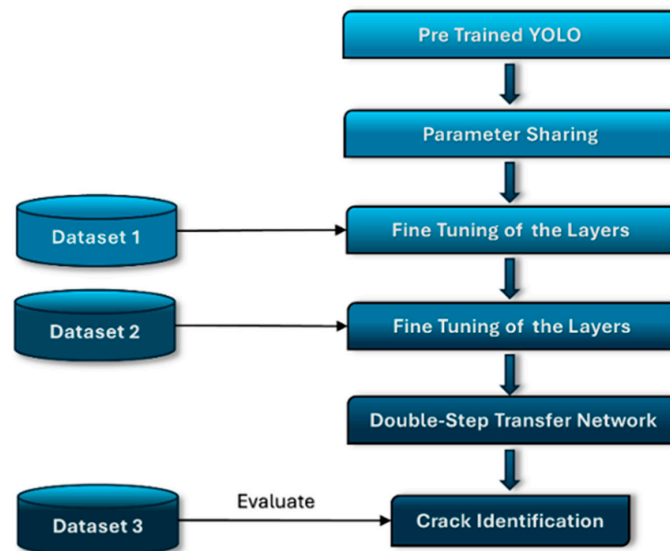


Figure 6. The illustration of the proposed model.

4.1. Training Phase

This study utilizes a concept of transfer learning-based deep neural networks for concrete crack detection. Different variants of you only look once (YOLO) are trained in two stages to create a transfer network for crack detection and segmentation in concrete structures. The YOLO models considered in this study are YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10. These different YOLO models have different variations in pre-trained models for classification, object detection, and segmentation tasks including YOLOv5n YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x, YOLOv6n YOLOv6s, YOLOv6m, YOLOv6l, YOLOv8n YOLOv8s, YOLOv8m, YOLOv8l, YOLOv8x, YOLOv9s, YOLOv9m, YOLOv9c, YOLOv9e, YOLOv10n YOLOv10s, YOLOv10m, YOLOv10l, and YOLOv10x [21]. The letters n, s, m, l, c, e, and x represent the sizes of these different models, i.e., nano, small, medium, large, compact, extended, and extra-large, respectively. The size of each model reflects the number of learnable parameters it contains. As shown in Figure 5, different YOLO models are first trained on a dataset containing cracks in various concrete structures. During this process, a given YOLO model learns the abstract features associated with the cracks, which can identify cracks effectively. Next, the network is fine-tuned on a few samples taken from a test task. Once the training of the model is completed, inference of the cracks is performed on the unseen data provided in the third dataset.

4.2. Inference

Following the fine-tuning process, the model is evaluated on the unseen dataset containing concrete cracks. The evaluation is performed to check the model performance to infer cracks. The non-maximum suppression (NMS) is used in these YOLO models during the inference process to eliminate undesired bounding boxes. The threshold set for both IoU and confidence during the experiment is 0.5.

4.3. Segmentation

In the next phase, segmentation masks are generated from the predictions made by the model. The segmentation process is repeated for all the predictions made by the model. Afterward, the extracted masks are visually assessed in comparison to the true labels to check the efficacy of the whole process.

5. Data Description

The digital images are obtained from public repositories that contain images of cracks in various concrete structures including, walls, roads, and pavements [26,27]. The details of these two datasets are provided in the following subsections.

5.1. Structural Defect Network 2018 Dataset

A publicly available dataset known as the structural defects network 2018 (SD-NET2018) dataset is used initially to train the network to explore high-level abstract features of the concrete cracks [33]. The dataset contains 56,000 labeled images, i.e., with and without crack images from different concrete structures, such as bridge decks, pavements, and walls. It is a challenging dataset for the crack detection process as it contains obstructions such as background shadows, debris, holes, uneven surfaces, and edges. The size of the cracks in the dataset ranges from 0.06 mm to 25 mm. To train the network for abstract feature exploration, the images are split into two subsets, namely with and without cracks subsets. There are 28,000 images in each category and each image has a resolution of 256×256 pixels.

5.2. Deep Crack Dataset

A detailed dataset containing 6315 images publicly available at [34] is used as a test set for crack detection and the segmentation process. The dataset is composed of images containing cracks from a variety of concrete structures including bridges, roadways, and buildings. From the dataset, 25% of the images were used to perform final fine-tuning of the transfer network. The remaining 75% of the images were utilized as unseen test data to infer cracks.

5.3. Datasets Preparation for the Experiment

In this study, transfer networks are developed by fine-tuning the top layers of different pretrained YOLO models using two distinct datasets. In the first stage, a dataset containing crack and non-crack images of concrete structures is utilized so that the network may learn the fine details regarding concrete cracks, which can be helpful in differentiating images containing cracks from non-crack images. From this dataset, 75 percent of the images are used to fine-tune the network, and the rest of the images are used as a validation subset. In the second stage, a different dataset is used to further fine-tune the network. This dataset is further subdivided into two subsets, i.e., Dataset 2 and Dataset 3. Dataset 2 is used for fine-tuning the network and updating the parameters of the network according to the target task, and Dataset 3 is used as a test subset. The model is again fine-tuned on a dataset containing images of cracks only, as the target task is to infer cracks in the concrete structures. Once the transfer network is ready, it is tested on a subset of unseen images from the second dataset. The organization of subsets used in the experiment is elaborated in Table 1.

Table 1. The details of the datasets used for transfer learning phases.

Dataset	Total No. of Samples for Training and Validation	Finetuning Phase	Validation Phase	Testing Phase
Dataset-1	44,800	33,600	11,200	--
Dataset-2	268	134	134	--
Dataset-3	268	--	--	268

6. Results Analysis

This study presents a study on the performance of different transfer networks developed using YOLO models for crack detection and segmentation. The suggested approach

consists of three phases, (1) the training of the models, (2) the crack inference in the unseen test images, and (3) the segmentation mask extraction. The proposed model is tested on the publicly available dataset as described in Section 4.

6.1. Evaluation Metrics

The mean average precision (mAP) and F1, which are derived from the precision and recall scores, are used as performance evaluation metrics of the model. The expressions for these metrics are given in Table 2.

Table 2. The performance evaluation metrics used in this work.

Evaluation Metrics	Formulation
Mean average precision	$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$ where N is the total number of classes and AP_i indicates the average precision of the i th class.
Precision	$\frac{\text{True positive Samples}}{\text{True positive Samples} + \text{False Positive Samples}} \times 100$
Recall	$\frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \times 100$
F1-score	$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \times 100$

6.2. Hyperparameters Selection

In this paper, during the hyperparameter tuning process for optimizing the ML model, we considered the requirements of loss convergence, computational efficiency, and generalization for YOLOv5, YOLOv6, YOLOv8, YOLOv9, and YOLOv10. Table 3 presents the hyperparameter values used in the experiments, specifically listing the values for the largest model in each category, as these required the highest number of epochs to minimize the objective function. The SiLU activation function was chosen for all YOLO models due to its superior gradient handling and suitability for object detection tasks. To achieve faster and more stable training, the learning rate was set at 0.001 and the batch size was set at 16 to minimize the computational costs. The number of epochs was set to 100 in the first transfer stage and 50 in the second, based on the model's convergence rate. These hyperparameters were tuned to optimize performance, generalization, and computational complexity.

Table 3. The hyperparameters selection for different models.

Model	Activation Function	Epochs to Minimize Loss (First Transfer Stage)	Epochs to Minimize Loss (Second Transfer Stage)	Optimal Learning Rate	Batch Size
YOLOv5x	SiLU	100	50	0.001	16
YOLOv6l	SiLU	100	50	0.001	16
YOLOv8x	SiLU	100	50	0.001	16
YOLOv9e	SiLU	100	50	0.001	16
YOLOv10x	SiLU	100	50	0.001	16

6.3. Transfer Network Development

To assess the robustness of the YOLO framework in effectively extracting useful information for crack identification and segmentation, different transfer learning networks are implemented in this work. A transfer-based approach is adopted due to its two main advantages over conventional model development. First, instead of building a network from scratch, the parameters of a pre-trained network can be easily updated by fine-tuning the top layers while freezing the rest. Second, this approach significantly reduces the time required to update the learnable parameters of a model according to the target task, as shown in Figure 7.

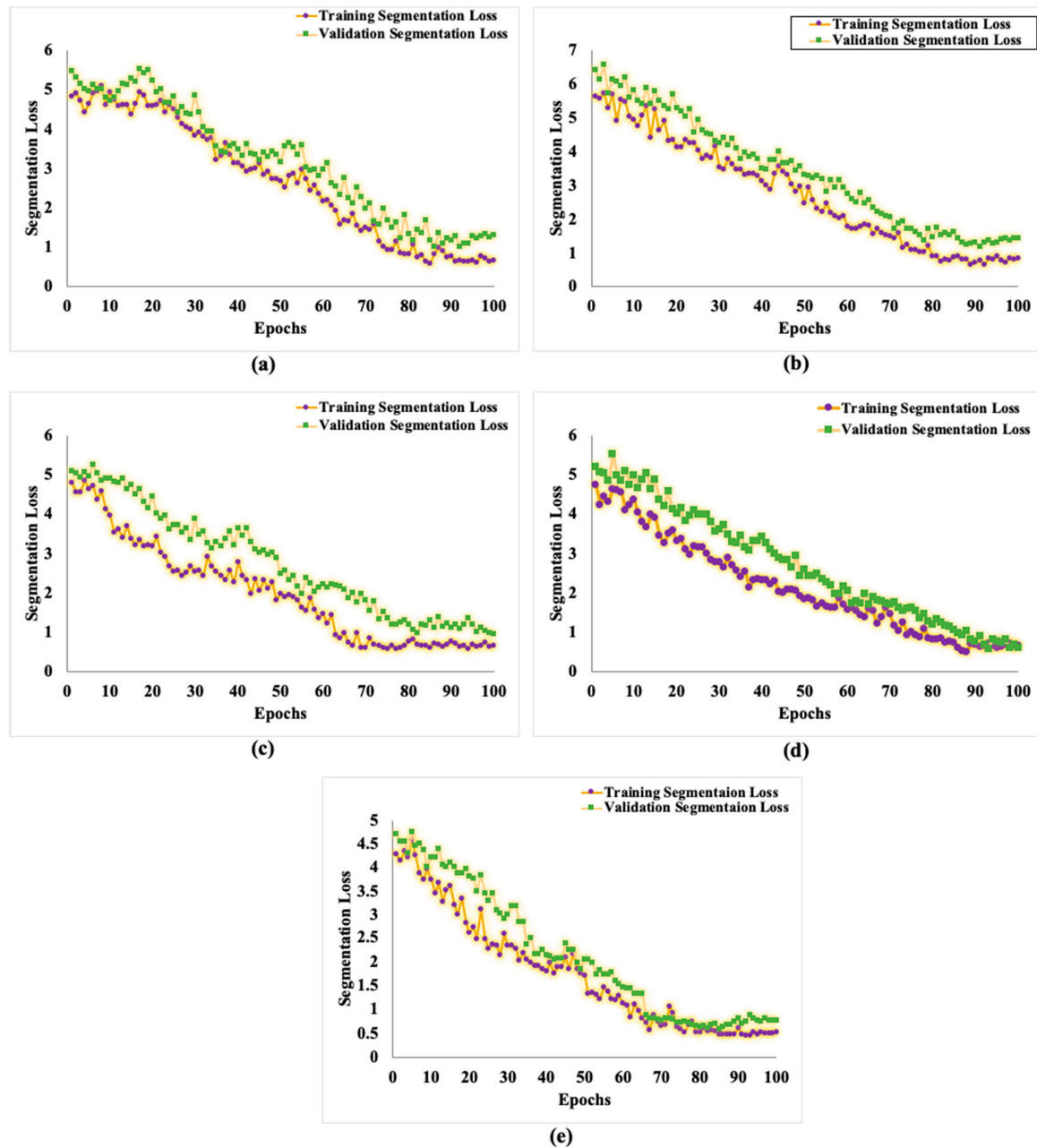


Figure 7. The training and validation segmentation loss curves of the various YOLO-based transfer networks in the first step, (a) YOLOv5x, (b) YOLOv6l, (c) YOLOv8x, (d) YOLOv9e, and (e) YOLOv10x.

In the figure, the convergence of loss curves for different YOLO models, i.e., YOLOv5x, YOLOv6l, YOLOv8x, YOLOv9e, and YOLOv10x, is presented. For each model category, the largest variant (in terms of parameter size) is considered, as it requires the greatest number of epochs to fully update the learnable parameters. It is observed that for all the models, the loss in both the training and validation phases converged to its minimum within 100 epochs. Furthermore, YOLOv10x significantly minimizes the objective losses in both phases compared to the other models. It is followed by YOLOv9e and YOLOv8x, respectively. YOLOv6 is the least effective in adequately reducing loss values in both phases.

In the second stage, the models required fewer epochs to converge the loss to minimal values when fine-tuned on the second dataset. This is due to the pre-learned salient features related to concrete cracks in the first phase of the transfer learning process. Therefore, as shown in Figure 8, the models took fewer epochs to update their learnable parameters according to the target task. The adequate convergence of losses in both phases indicates

that the models were appropriately fine-tuned and ready to be tested on an unseen dataset. This assumption is supported by the mAP and F1 values listed in Table 2.

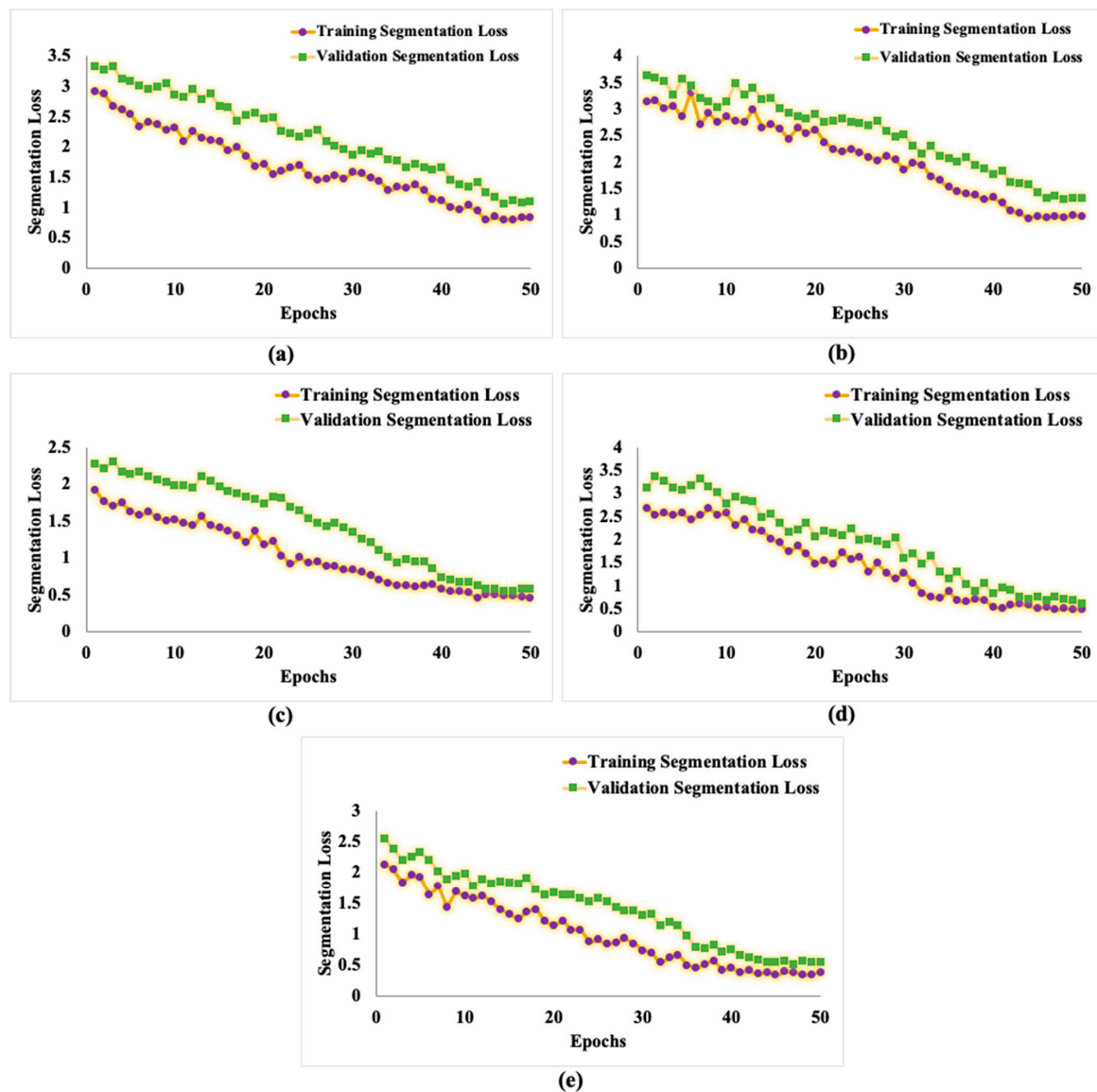


Figure 8. The training and validation segmentation loss curves of the various YOLO-based transfer networks in the second step, (a) YOLOv5x, (b) YOLOv6l, (c) YOLOv8x, (d) YOLOv9e, and (e) YOLOv10x.

It can be observed that the best mAP values during training and validation in the first phase of transfer learning were 0.86 and 0.83, respectively. In the second phase, the highest mAP values were 0.89 (training) and 0.85 (validation).

6.4. Model Evaluation on Unseen Dataset

Table 4 presents the evaluation metrics of different YOLO-based transfer models during the training and validation phases. YOLOv10x and YOLOv9e exhibit similar performance, followed by YOLOv8x. The highest mAP and F1 values during training and validation are achieved by YOLOv10x, with 76.19% and 74.52%, respectively. YOLOv9e follows closely with mAP values of 76.08% and 73.99% for training and validation, respectively. YOLOv8x achieved 75.26% (training) and 73.99% (validation). The lowest performance was observed with YOLOv5n, which had mAP values of 64% and 60.5% during training and validation.

Table 4. The performance metrics for the various YOLO models during the training and validation phases.

Model	Train		Validation		Parameters (Millions)	Inference Time per Image (Milliseconds)
	mAP (%)	F1 Score (%)	mAP (%)	F1 Score (%)		
YOLOv5n	66	73.64	62	72	1.9	7.3
YOLOv6n	64	69.72	60.5	68	4.7	11
YOLOv8n	66.3	75.86	62	74.35	3.2	9
YOLOv10n	67.8	76.94	63.4	75.20	2.3	6
YOLOv5s	67.74	75.17	66.39	73.58	7.2	10
YOLOv6s	65.79	72.82	61.37	71.25	18.5	13
YOLOv8s	68.13	79.71	66.51	77.83	11.2	12
YOLOv9s	68.46	79.82	67.38	78.14	7.2	9.7
YOLOv10s	69.58	80.48	67.92	78.36	7.2	8.5
YOLOv5m	69.98	78.35	69.04	75	21.2	13.5
YOLOv6m	67.24	75.83	66.56	73.47	34.9	14.2
YOLOv8m	71.40	81.80	69.44	78.58	25.9	18
YOLOv9m	70.82	79.96	69.51	78.39	20.1	12.8
YOLOv10m	72.48	83.47	70.82	79.20	15.4	10
YOLOv5l	71.36	80.90	69.86	77.63	46.5	17
YOLOv6l	69.78	79.62	68.92	74.01	59.6	15.8
YOLOv8l	73.75	82.39	72.62	79.25	43.7	23
YOLOv9c	73.94	83.62	71.85	79.03	25.5	18.24
YOLOv10l	74.06	84	72.15	80.1	24.4	15.7
YOLOv5x	74.37	82.94	72.24	79.58	86.7	21.1
YOLOv8x	75.26	83.49	73.61	81.50	68.23	32
YOLOv9e	76.08	84.26	73.99	82	58.1	25.63
YOLOv10x	76.19	84.50	74.52	82.73	29.5	19.5

A similar trend is evident in the F1 scores, with YOLOv10x scoring the highest, followed by YOLOv9e and YOLOv8x. YOLOv6n had the lowest F1 score. These performance indicators suggest that as the model size (i.e., the number of learnable parameters) increases, the performance also improves. Therefore, if there are no hardware resource limitations; models with larger capacities can be used to achieve the highest detection accuracy.

The table also provides the inference speed per image for these transfer models. YOLOv10x outperformed other models with a speed advantage of at least 1.2 milliseconds per image. The superior performance of YOLOv10x compared to other variants can be attributed to its larger architecture, which allows for more learnable parameters, improving its ability to capture complex features. Additionally, its more advanced optimization strategies and improved architecture contribute to better generalization and faster inference times.

Additionally, a visual inspection of crack detection and segmentation performance for each model type is provided in Figures 9–13. These results indicate that all models could easily infer macro cracks. However, complications arise in cases with complex crack distribution patterns, micro-cracks, or visually complex scenarios. In such cases, advanced YOLO architectures (YOLOv8, YOLOv9, YOLOv10) show relatively better crack detection capabilities. This can be verified by the inference results shown in Figures 11–13. For instance, in Figures 9c and 10c, YOLOv5 and YOLOv6 struggle with micro-cracks

and complex distribution patterns, resulting in segmentation masks that deviate from the true masks.

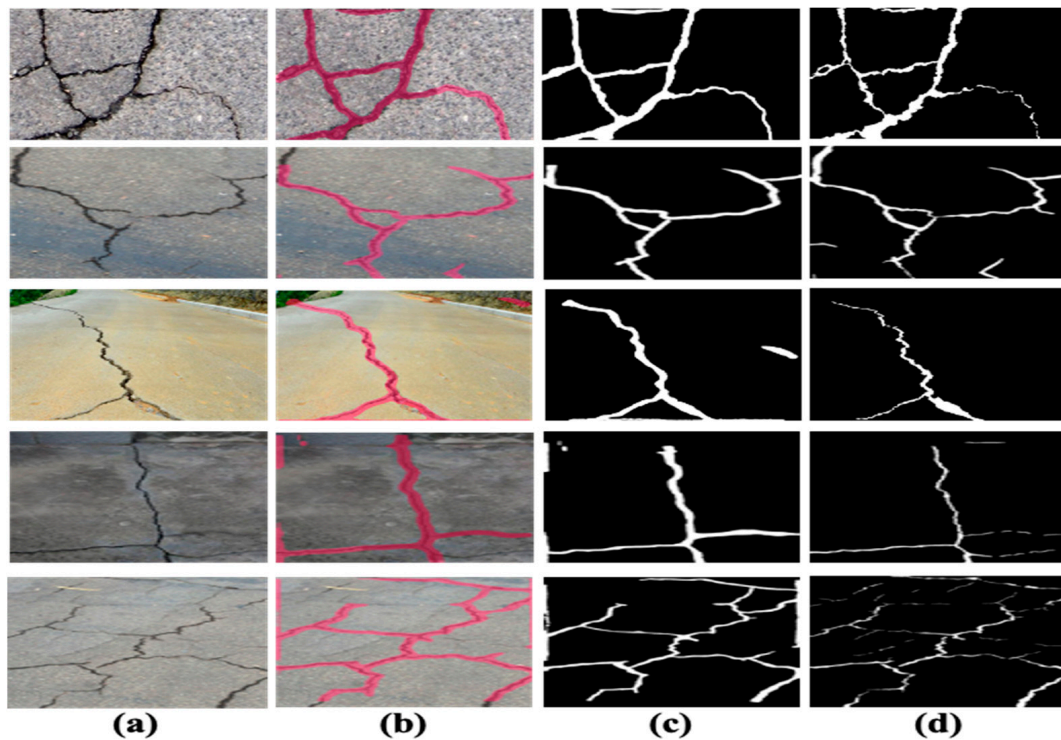


Figure 9. The inference and segmentation results of the YOLOv5x model along with the original image and true labels, (a) original images, (b) inferred cracks, (c) segmented binary masks, and (d) true binary masks.

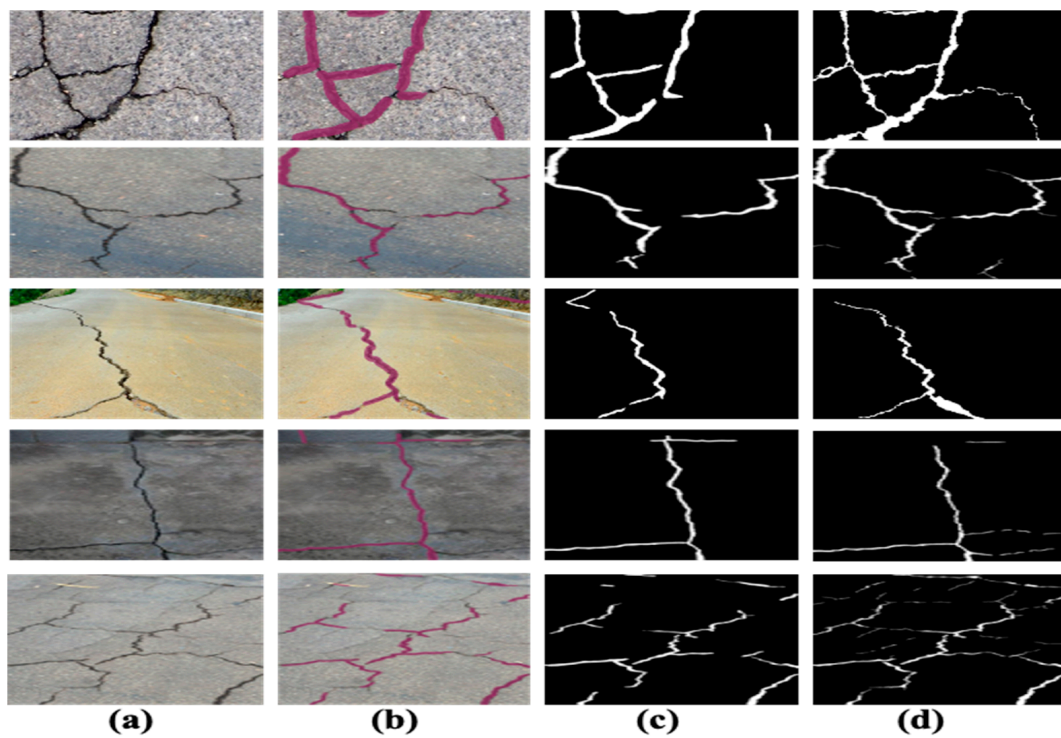


Figure 10. The inference and segmentation results of the YOLOv6l model along with the original image and true labels, (a) original images, (b) inferred cracks, (c) segmented binary masks, and (d) true binary masks.

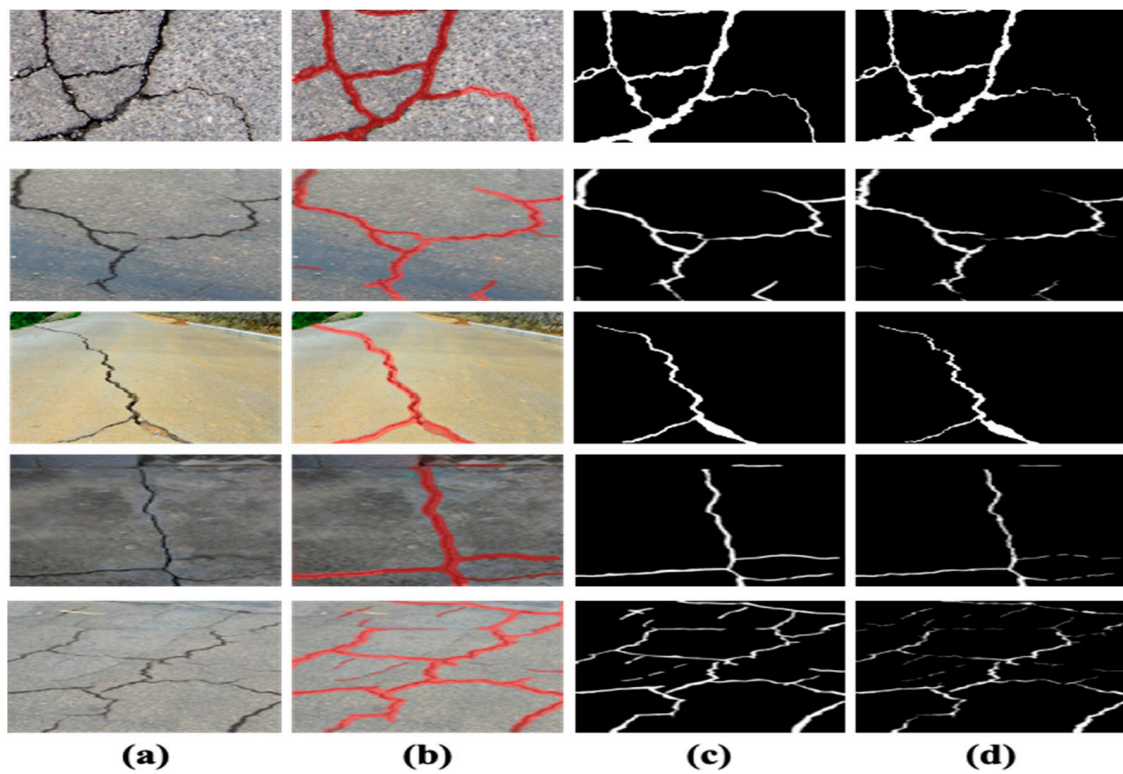


Figure 11. The inference and segmentation results of the YOLOv8x model along with the original image and true labels, (a) original images, (b) inferred cracks, (c) segmented binary masks, and (d) true binary masks.

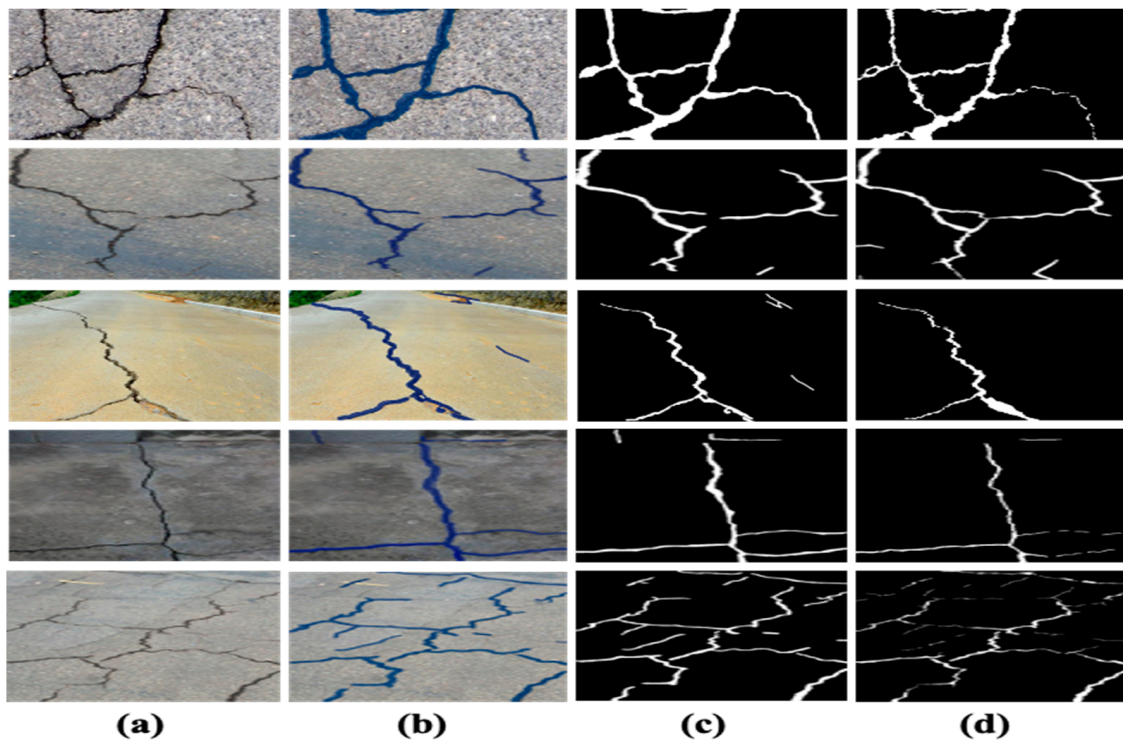


Figure 12. The inference and segmentation results of the YOLOv9e model along with the original image and true labels, (a) original Images, (b) inferred cracks, (c) segmented binary masks, and (d) true binary masks.

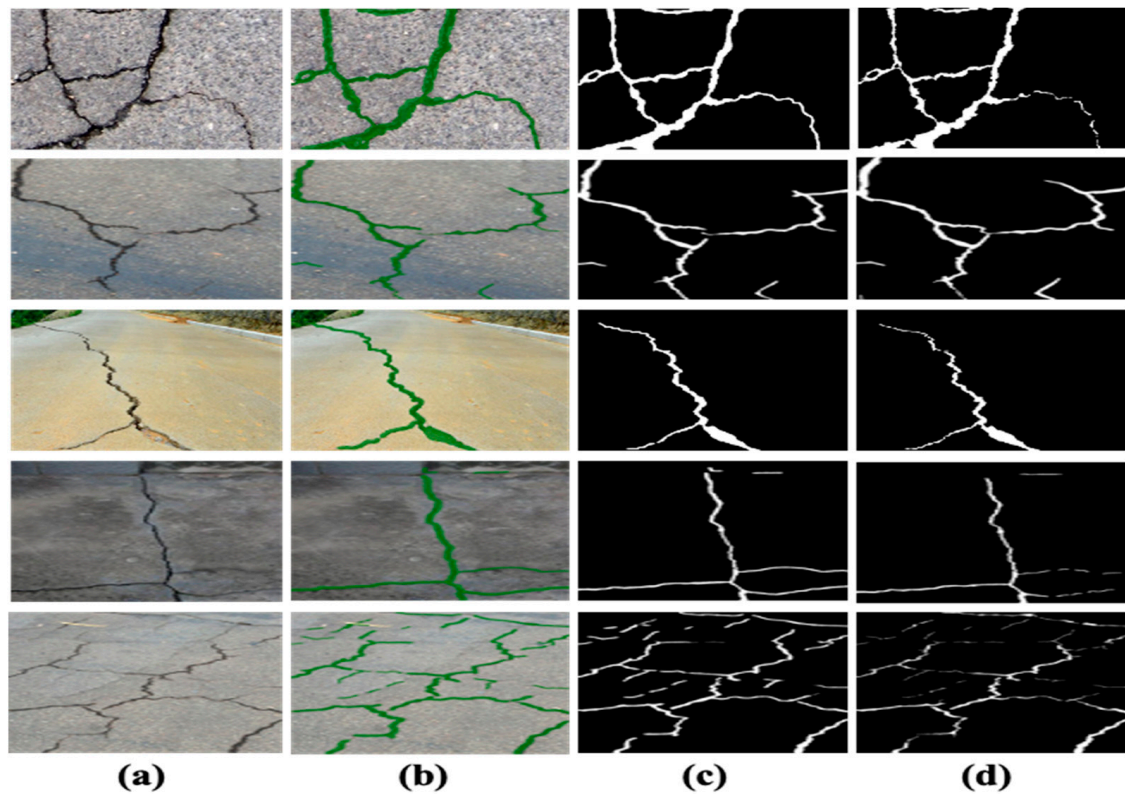


Figure 13. The inference and segmentation results of the YOLOv10x model along with the original image and true labels, (a) original Images, (b) inferred cracks, (c) segmented binary masks, and (d) true binary masks.

7. Conclusions

The proposed models were evaluated on two distinct datasets, each containing images of cracks with complex backgrounds, which more closely resemble real-world conditions. The results showed the robustness of the models in accurately detecting and segmenting cracks even in challenging environments. Notably, the YOLOv10x model demonstrated superior performance in terms of both mean average precision (mAP) and inference speed, making it a practical solution for real-time crack detection.

In conclusion, the transfer learning approach applied in this study not only enhances detection precision but also reduces computational overhead. With low inference times and high mAP values, the models presented here are well-suited for deployment in real-time monitoring systems for concrete structures. Additionally, this model has the potential to be adapted for broader applications, such as cracking detection in asphalt pavements, which experience similar degradation processes. By fine-tuning the model to account for the specific characteristics of asphalt, such as its texture, color variation, and crack patterns, it could be effectively applied to pavement monitoring. These findings offer valuable insights for future research, particularly in selecting appropriate YOLO architectures for crack detection and segmentation in challenging real-world scenarios.

Future augmentations can refine the presented models for edge devices and some GPU devices using model pruning, the quantization process, and knowledge distillation. Furthermore, to improve the smooth run of the models, utilizing tricks such as TensorRT and/or mixed-precision inference or utilizing lightweight structures like the backbone of the architecture would be useful. Specifying hardware requirements and aiming at memory and computational efficiency on new datasets would be an imminent task for practitioners who want to adopt these models in realistic applications.

Author Contributions: Conceptualization, M.S. and M.A.; methodology, M.S., M.A. and J.-M.K.; software, M.S. and M.A.; validation, M.S., M.A. and J.-M.K.; formal analysis, M.S., M.A. and J.-M.K.; investigation, M.S. and J.-M.K.; resources, M.S. and J.-M.K.; data curation, M.S. and M.A.; writing—original draft preparation, M.S., M.A. and J.-M.K.; writing—review and editing, M.S., M.A. and J.-M.K.; visualization, M.S. and M.A.; supervision, M.S. and J.-M.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Ulsan City and Electronics and Telecommunications Research Institute (ETRI) grant funded by the Ulsan City [24AB1600, the development of intelligentization technology for the main industry for manufacturing innovation and Human-mobile-space autonomous collaboration intelligence technology development in industrial sites]. This work was also supported by the Korea Institute of Energy Technology Evaluation and Planning (KETEP) grant funded by the Korean government (MOTIE) ('RS-2024-00449107', 'Development of Flexible Pipe and Connector for Hydrogen gas').

Data Availability Statement: The data are available upon request.

Conflicts of Interest: Author Jong-Myon Kim was employed by the company PD Technology Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflicts of interest.

References

1. Janev, D.; Nakov, D.; Arangjelovski, T. Concrete for Resilient Infrastructure: Review of Benefits, Challenges and Solutions. In Proceedings of the 20th International Symposium of MASE: Macedonian Association of Structural Engineers (MASE), Skopje, North Macedonia, 28–29 September 2023; pp. 208–219.
2. Asmara, Y.P. *Concrete Reinforcement Degradation and Rehabilitation: Damages, Corrosion and Prevention*; Springer Nature: Singapore, 2023; ISBN 9819959330.
3. Olurotimi, O.J.; Yetunde, O.H.; Akah, A.R.C.U. Assessment of the Determinants of Wall Cracks in Buildings: Investigating the Consequences and Remedial Measure for Resilience and Sustainable Development. *Int. J. Adv. Educ. Manag. Sci. Technol.* **2023**, *6*, 121–132.
4. Hu, Y.; Sreeram, A.; Airey, G.D.; Li, B.; Si, W.; Wang, H. Comparative Analysis of Time Sweep Testing Evaluation Methods for the Fatigue Characterisation of Aged Bitumen. *Constr. Build. Mater.* **2024**, *432*, 136698. [[CrossRef](#)]
5. Adwani, D.; Pipintakos, G.; Mirwald, J.; Wang, Y.; Hajj, R.; Guo, M.; Liang, M.; Jing, R.; Varveri, A.; Zhang, Y. Examining the Efficacy of Promising Antioxidants to Mitigate Asphalt Binder Oxidation: Insights from a Worldwide Interlaboratory Investigation. *Int. J. Pavement Eng.* **2024**, *25*, 2332363. [[CrossRef](#)]
6. Lin, H.; Han, Y.; Liang, S.; Gong, F.; Han, S.; Shi, C.; Feng, P. Effects of Low Temperatures and Cryogenic Freeze-Thaw Cycles on Concrete Mechanical Properties: A Literature Review. *Constr. Build. Mater.* **2022**, *345*, 128287. [[CrossRef](#)]
7. Zhang, W.; Pi, Y.; Kong, W.; Zhang, Y.; Wu, P.; Zeng, W.; Yang, F. Influence of Damage Degree on the Degradation of Concrete under Freezing-Thawing Cycles. *Constr. Build. Mater.* **2020**, *260*, 119903. [[CrossRef](#)]
8. Feng, G.; Zhu, D.; Guo, S.; Rahman, M.Z.; Jin, Z.; Shi, C. A Review on Mechanical Properties and Deterioration Mechanisms of FRP Bars under Severe Environmental and Loading Conditions. *Cem. Concr. Compos.* **2022**, 104758. [[CrossRef](#)]
9. Wang, J.; Ueda, T. A Review Study on Unmanned Aerial Vehicle and Mobile Robot Technologies on Damage Inspection of Reinforced Concrete Structures. *Struct. Concr.* **2023**, *24*, 536–562. [[CrossRef](#)]
10. Eslamlou, A.D.; Ghaderiaram, A.; Schlangen, E.; Fotouhi, M. A Review on Non-Destructive Evaluation of Construction Materials and Structures Using Magnetic Sensors. *Constr. Build. Mater.* **2023**, *397*, 132460. [[CrossRef](#)]
11. Lattanzi, D.; Miller, G. Review of Robotic Infrastructure Inspection Systems. *J. Infrastruct. Syst.* **2017**, *23*, 4017004. [[CrossRef](#)]
12. Vijayan, V.; Joy, C.M.; Shailesh, S. A Survey on Surface Crack Detection in Concretes Using Traditional, Image Processing, Machine Learning, and Deep Learning Techniques. In Proceedings of the 2021 International Conference on Communication, Control and Information Sciences (ICCIsc), Idukki, India, 16–18 June 2021; IEEE: Piscataway, NJ, USA, 2021; Volume 1, pp. 1–6.
13. Ai, D.; Jiang, G.; Lam, S.-K.; He, P.; Li, C. Computer Vision Framework for Crack Detection of Civil Infrastructure—A Review. *Eng. Appl. Artif. Intell.* **2023**, *117*, 105478. [[CrossRef](#)]
14. Liu, C.; Du, Y.; Yue, G.; Li, Y.; Wu, D.; Li, F. Advances in Automatic Identification of Road Subsurface Distress Using Ground Penetrating Radar: State of the Art and Future Trends. *Autom. Constr.* **2024**, *158*, 105185. [[CrossRef](#)]
15. Hussein, R.; Etete, B.; Mahdi, H.; Al-Shukri, H. Detection and Delineation of Cracks and Voids in Concrete Structures Using the Ground Penetrating Radar Technique. *J. Appl. Geophys.* **2024**, *226*, 105379. [[CrossRef](#)]
16. Guo, J.; Liu, P.; Xiao, B.; Deng, L.; Wang, Q. Surface Defect Detection of Civil Structures Using Images: Review from Data Perspective. *Autom. Constr.* **2024**, *158*, 105186. [[CrossRef](#)]
17. Bhattacharya, G.; Mandal, B.; Puan, N.B. Interleaved Deep Artifacts-Aware Attention Mechanism for Concrete Structural Defect Classification. *IEEE Trans. Image Process.* **2021**, *30*, 6957–6969. [[CrossRef](#)] [[PubMed](#)]

18. Zhao, W.; Liu, Y.; Zhang, J.; Shao, Y.; Shu, J. Automatic Pixel-level Crack Detection and Evaluation of Concrete Structures Using Deep Learning. *Struct. Control Health Monit.* **2022**, *29*, e2981. [\[CrossRef\]](#)
19. Zhang, J.; Cai, Y.-Y.; Yang, D.; Yuan, Y.; He, W.-Y.; Wang, Y.-J. MobileNetV3-BLS: A Broad Learning Approach for Automatic Concrete Surface Crack Detection. *Constr. Build. Mater.* **2023**, *392*, 131941. [\[CrossRef\]](#)
20. Laxman, K.; Tabassum, N.; Ai, L.; Cole, C.; Ziehl, P. Automated Crack Detection and Crack Depth Prediction for Reinforced Concrete Structures Using Deep Learning. *Constr. Build. Mater.* **2023**, *370*, 130709. [\[CrossRef\]](#)
21. Wang, L. Automatic Detection of Concrete Cracks from Images Using Adam-SqueezeNet Deep Learning Model. *Frat. Ed Integrità Strutt.* **2023**, *17*, 289–299. [\[CrossRef\]](#)
22. Mishra, V.; Kane, L. A Survey of Designing Convolutional Neural Network Using Evolutionary Algorithms. *Artif. Intell. Rev.* **2023**, *56*, 5095–5132. [\[CrossRef\]](#)
23. Sohaib, M.; Jamil, S.; Kim, J.-M. An Ensemble Approach for Robust Automated Crack Detection and Segmentation in Concrete Structures. *Sensors* **2024**, *24*, 257. [\[CrossRef\]](#)
24. Sohaib, M.; Hasan, M.J.; Shah, M.A.; Zheng, Z. A Robust Self-Supervised Approach for Fine-Grained Crack Detection in Concrete Structures. *Sci. Rep.* **2024**, *14*, 12646. [\[CrossRef\]](#) [\[PubMed\]](#)
25. Models Supported by Ultralytics—Ultralytics YOLO Docs. Available online: <https://docs.ultralytics.com/models/> (accessed on 30 January 2024).
26. Nigar, N.; Muhammad Faisal, H.; Kashif Shahzad, M.; Islam, S.; Oki, O. An Offline Image Auditing System for Legacy Meter Reading Systems in Developing Countries: A Machine Learning Approach. *J. Electr. Comput. Eng.* **2022**, *2022*, 4543530. [\[CrossRef\]](#)
27. Li, C.; Li, L.; Jiang, H.; Weng, K.; Geng, Y.; Li, L.; Ke, Z.; Li, Q.; Cheng, M.; Nie, W. YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. *arXiv* **2022**, arXiv:2209.02976.
28. YOLOv8—Ultralytics YOLOv8 Docs. Available online: <https://docs.ultralytics.com/models/yolov8/> (accessed on 30 January 2024).
29. Zheng, Z.; Wang, P.; Liu, W.; Li, J.; Ye, R.; Ren, D. Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. In Proceedings of the AAAI conference on artificial intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 12993–13000.
30. Li, X.; Wang, W.; Wu, L.; Chen, S.; Hu, X.; Li, J.; Tang, J.; Yang, J. Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection. *Adv. Neural. Inf. Process. Syst.* **2020**, *33*, 21002–21012.
31. Wang, C.-Y.; Yeh, I.-H.; Liao, H.-Y.M. Yolov9: Learning What You Want to Learn Using Programmable Gradient Information. *arXiv* **2024**, arXiv:2402.13616.
32. Wang, A.; Chen, H.; Liu, L.; Chen, K.; Lin, Z.; Han, J.; Ding, G. Yolov10: Real-Time End-to-End Object Detection. *arXiv* **2024**, arXiv:2405.14458.
33. Maguire, M.; Dorafshan, S.; Thomas, R.J. *SDNET2018: A Concrete Crack Image Dataset for Machine Learning Applications*; Utah State University: Logan, UT, USA, 2018. [\[CrossRef\]](#)
34. Liu, Y.; Yao, J.; Lu, X.; Xie, R.; Li, L. DeepCrack: A Deep Hierarchical Feature Learning Architecture for Crack Segmentation. *Neurocomputing* **2019**, *338*, 139–153. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.